AI URBAN MICRO MOBILITY SIMULATOR

AI-Based E-Bike Route Optimization and Urban Mobility Simulation

DETAILED PROJECT APPROACH

A software-based simulation system that models an urban road network, generates alternative routes, predicts traffic conditions, estimates travel time and e-bike energy consumption, and selects an efficient route using AI-assisted multi-factor optimization.

Project Scope	Definition
Primary Goal	Simulate and optimize e-bike movement through an urban road network.
AI Role	Predict traffic/travel conditions and provide inputs to route optimization.
Routing	Dijkstra as the baseline; A* as an optional advanced routing algorithm.
Simulation	Controlled scenarios for traffic, travel time, energy, and route comparison.
Visualization	Interactive map showing source, destination, routes, and simulated e-bike movement.

Prepared as a technical project-approach document for the proposed academic project.

1. Project Overview

The AI Urban Micro Mobility Simulator is designed as a controlled software environment for studying how an e-bike should travel between two locations in an urban road network. The project is intentionally a simulator rather than a booking, rental, fleet-management, or navigation application. Its purpose is to demonstrate routing algorithms, traffic simulation, machine-learning prediction, energy estimation, and multi-objective optimization in one integrated workflow.

Core Question

Given the same source and destination, which route is most suitable for an e-bike when distance, travel time, traffic, and energy consumption are considered together?

Project Boundaries

- No user accounts, payments, booking, rental management, bike-station management, or fleet management.
- No dependence on physical e-bike hardware is required for the core simulator.
- The initial version can use synthetic road and traffic data so that experiments are repeatable.
- Real map data can be added later without changing the core optimization concept.

2. Problem Definition

A shortest-path solution is not always the best mobility solution for an e-bike. A short road segment can be heavily congested, while a slightly longer alternative may provide lower travel time and lower energy use. Similarly, road characteristics such as slope or repeated stops can influence energy consumption. Therefore, the simulator treats route selection as a multi-factor decision rather than a single-distance calculation.

Problem Statement

Develop a software simulator that models an urban road network and intelligently selects an efficient e-bike route by combining graph-based routing, simulated/predicted traffic, travel-time estimation, energy-consumption estimation, and multi-factor route optimization.

Objectives

- Create an urban road network represented as a weighted graph.
- Generate candidate routes between a selected source and destination.
- Calculate route distance and estimated travel time.
- Simulate traffic under different urban conditions.
- Train a machine-learning model to predict traffic or travel conditions.
- Estimate e-bike energy consumption for candidate routes.
- Normalize route factors and compute an overall route score.
- Select and simulate the optimal route.
- Visualize the route and simulation results on an interactive map.
- Compare traditional routing with the proposed AI-assisted optimization approach.

3. Overall System Approach

The project should be implemented incrementally. The routing engine is built first, then traffic and energy models are added, followed by machine-learning prediction and the final optimization layer. This avoids trying to debug the entire system at once.

End-to-End Pipeline

Stage	Input	Processing	Output
1. Network	Road/intersection data	Build graph	Urban road graph
2. Routing	Source + destination	Dijkstra/A*	Candidate routes
3. Traffic	Time + road data	Simulation + ML prediction	Traffic values
4. Time	Distance + speed + traffic	Effective-speed model	Travel time
5. Energy	Distance + traffic + road factors	Energy model	Energy estimate
6. Optimization	All route metrics	Normalize + weighted score	Best route
7. Simulation	Selected route	Step-by-step movement	Journey trace
8. Visualization	Routes + trace + metrics	Map + charts	Results dashboard

4. Urban Road Network Modeling

The urban environment is represented as a graph. Intersections or important locations are modeled as nodes, while road segments connecting them are modeled as edges. Each edge stores attributes that can affect routing and simulation.

Suggested Road Attributes

Attribute	Purpose
road_id	Unique road-segment identifier.
from_node / to_node	Defines graph connectivity.
distance_km	Physical road length.
speed_limit_kmph	Base speed used for travel-time estimation.
traffic_level	Current/simulated congestion value.
road_type	Classifies road characteristics.
slope	Optional factor for energy estimation.
intersection_count	Optional indicator of stops and delays.

Example Graph

A small network may contain nodes A, B, C, D, E and F. Multiple paths can connect A to F. Each path has different distance, speed, traffic and energy characteristics. This makes it possible to demonstrate that the shortest route is not necessarily the best route.

5. Route Generation Using Dijkstra and A*

Dijkstra's algorithm provides a clear traditional baseline for shortest-path routing. The project can initially assign distance as the edge weight. After the basic routing works, the edge weight can be changed to represent travel time or another normalized cost.

Dijkstra Implementation Approach

1. Create the graph from road data.
2. Assign an initial cost to every road edge.
3. Set the source node distance to zero and all other distances to infinity.
4. Use a priority queue to repeatedly select the node with the smallest known cost.
5. Relax neighboring edges when a lower-cost path is found.
6. Store predecessor nodes to reconstruct the final route.
7. Return the node sequence and total route cost.

A* Extension

A* can be introduced as an optional improvement using a heuristic based on the estimated distance from the current node to the destination. The same graph can therefore support an algorithm comparison without changing the rest of the system.

Important Academic Point

Dijkstra alone should not be presented as the AI component. It is the traditional graph-routing baseline. The AI component comes from machine-learning prediction and the use of predicted conditions in route optimization.

6. Traffic Simulation and AI Prediction

Traffic is a major variable because congestion can change both travel time and energy consumption. The first implementation can use synthetic traffic values so that the team can deliberately create low-, medium-, and high-traffic scenarios.

Traffic Representation

- Categorical: Low, Medium, High.
- Continuous: a normalized traffic value between 0.0 and 1.0.
- Time-dependent: different values for morning, afternoon, evening, and peak periods.

Machine-Learning Pipeline

1. Create a historical/synthetic traffic dataset.
2. Clean and prepare the data.
3. Select features such as time of day, day of week, road ID, distance, road type, previous traffic and speed.
4. Split data into training and testing sets.
5. Train a beginner-friendly model such as Random Forest Regression or Classification.
6. Evaluate the model using suitable metrics.
7. Save the trained model.
8. During simulation, pass current road/time conditions to the model.
9. Use the predicted traffic value as an input to travel-time and route scoring.

Example Dataset Fields

Feature	Example	Role
hour	08	Captures peak-hour effects.
day_of_week	Monday	Captures weekly patterns.
road_id	R12	Identifies road segment.
distance_km	1.8	Road length.
road_type	Urban	Road category.
previous_traffic	0.65	Historical/previous condition.
predicted_traffic	0.72	Target/output.

7. Travel-Time and E-Bike Energy Models

7.1 Travel-Time Estimation

For a simple baseline, travel time can be calculated from distance and effective speed. Traffic reduces effective speed. The exact reduction function should be calibrated during testing rather than treated as a real-world universal constant.

Base relationship: Time = Distance / Speed

For example, an effective-speed function can reduce the road's speed according to a normalized traffic factor. The simulator can then sum the travel time of all edges on a candidate route.

7.2 Energy Consumption

The first version should use a transparent baseline energy model. More detailed factors can be added after the baseline is validated.

Baseline relationship: Energy = Distance × Base Consumption

An enhanced model can include traffic, slope, stops, and acceleration/deceleration as adjustment factors. This allows the project to explain why two routes with similar distance can have different energy requirements.

Example conceptual model

Energy ≈ Distance × BaseConsumption × TrafficFactor × SlopeFactor × StopFactor

The coefficients should be treated as simulation parameters unless supported by measured data. This keeps the model honest and reproducible.

8. Multi-Factor Route Optimization

Each candidate route is evaluated using several criteria. Because distance, time, traffic, and energy use have different units, they should be normalized before being combined.

$$\text{Route Score} = W_1 \times \text{Distance_norm} + W_2 \times \text{Time_norm} + W_3 \times \text{Traffic_norm} + W_4 \times \text{Energy_norm}$$

Example Starting Weights

Factor	Weight	Reason
Distance	0.40	Keeps route length important.
Travel Time	0.30	Penalizes slow/congested routes.
Traffic	0.20	Reduces exposure to congestion.
Energy	0.10	Adds e-bike efficiency consideration.

These are initial project parameters, not measured optimal weights. The team can run sensitivity experiments by changing the weights and observing how the selected route changes.

Route Decision

- Generate several feasible candidate routes.
- Calculate all route metrics.
- Normalize each metric.
- Apply the selected weights.
- Choose the route with the lowest overall cost/score.
- Store both the selected route and the alternatives for comparison.

9. Simulation Engine

The simulation engine represents the e-bike travelling through the selected sequence of road segments. It does not require physical GPS hardware. The simulator can update the journey at fixed time or distance intervals.

Simulation State

Variable	Meaning
current_node	Current position in the road graph.
current_edge	Road segment currently being traversed.
distance_travelled	Cumulative distance.
elapsed_time	Cumulative simulated journey time.
energy_used	Cumulative estimated energy.
traffic_level	Current traffic condition.
status	Running, paused, completed, or stopped.

Simulation Steps

1. Load the selected route.
2. Initialize e-bike position at the source.
3. Read road conditions for the current segment.
4. Estimate effective speed and travel time.
5. Update distance and energy consumption.
6. Move the simulated position along the road.
7. Continue until the destination is reached.
8. Record the journey metrics.
9. Display the final result and compare it with alternative routes.

10. Map Visualization and User Interface

The frontend should make the simulation understandable to an evaluator. A map-based interface can show the urban network, source and destination, candidate routes, selected route, and the simulated e-bike position.

Recommended Interface

- Source selector and destination selector.
- Simulation condition controls such as traffic scenario and time of day.
- Buttons for route generation, optimization, start/pause/reset simulation.
- Map with source and destination markers.
- Different visual treatment for candidate and selected routes.
- Live metric panel: distance, elapsed time, traffic, energy.
- Route comparison table and simple charts after simulation.

Suggested Technology

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	Interface and interaction.
Map	Leaflet.js + OpenStreetMap	Map and route visualization.
Backend	Python + FastAPI	Routing, simulation and API services.
Database	SQLite	Road, traffic and simulation records.
Data/ML	Pandas + Scikit-learn	Data processing and traffic model.
Charts	Chart.js	Result comparison.

11. Backend and Module Design

A modular backend makes the project easier to develop, test, and explain during a viva. Each major calculation should be separated into a small Python module rather than placing the entire project in one file.

Recommended Structure

```
backend/
├── main.py
├── algorithms/
│   ├── dijkstra.py
│   └── astar.py
├── ai/
│   ├── train_model.py
│   └── traffic_prediction.py
├── simulation/
│   ├── traffic.py
│   ├── energy.py
│   └── simulator.py
├── optimization/
│   └── route_optimizer.py
├── database/
│   └── database.py
└── data/
    ├── roads.csv
    └── traffic.csv

frontend/
├── index.html
├── style.css
└── app.js

models/
└── traffic_model.pkl

tests/
requirements.txt
README.md
```

Suggested API Endpoints

Endpoint	Purpose
GET /roads	Return road-network data.
POST /route	Generate a route between source and destination.
POST /optimize	Evaluate candidate routes and return the optimal route.
POST /simulate	Run a journey simulation.
GET /results	Return stored simulation and comparison results.

12. Data and Database Design

The initial project can be built with CSV files for rapid experimentation and later connected to SQLite for persistent simulation records. This keeps the development path simple.

Roads Table

Field	Example
road_id	R01
from_node	A
to_node	B
distance_km	1.20
speed_limit	35
road_type	arterial
slope	0.03

Traffic Table

Field	Purpose
traffic_id	Unique observation.
road_id	Road being measured/simulated.
timestamp/hour	Time context.
traffic_level	Observed/simulated traffic value.
speed	Observed/simulated effective speed.

Simulation Table

- simulation_id, source, destination, scenario, selected_route, distance, time, energy, traffic, score, created_at.

13. Evaluation Strategy

Evaluation should demonstrate not only that the route can be found, but that the proposed approach changes route selection logically under different conditions. Results should be recorded for reproducibility.

Key Metrics

Metric	What it Shows
Route distance	Physical route length.
Travel time	Impact of speed and congestion.
Traffic exposure	How much congestion affects the route.
Energy consumption	Estimated e-bike energy requirement.
Overall score	Combined optimization objective.
Computation time	Routing/optimization efficiency.

Test Scenarios

1. Low traffic: verify that shortest/efficient routes remain competitive.
2. Heavy traffic on the shortest route: verify that the optimizer can select a better alternative.
3. High-slope route: verify that energy-aware scoring can penalize the route.
4. Peak-hour scenario: verify the effect of predicted/simulated traffic.
5. Low-energy scenario: test whether route weighting can prioritize lower energy use.
6. Weight sensitivity: change optimization weights and observe route-selection changes.

14. Traditional Routing vs Proposed Approach

Aspect	Traditional Baseline	Proposed Simulator
Primary objective	Shortest/lowest graph cost	Multi-factor e-bike route efficiency
Routing	Dijkstra	Dijkstra/A* + optimization
Traffic	Static/limited weight	Simulated and ML-predicted conditions
Energy	Usually not central	Explicit energy model
Experiments	Not the main purpose	Controlled scenarios
Evaluation	Route result	Distance/time/traffic/energy/score comparison
Output	Path	Optimal path + simulation + analytics

Difference from a General Navigation App

Navigation systems answer: “How do I get from A to B?”
This simulator studies: “Under different urban conditions, which route is most efficient for an e-bike, and how does the simulated journey behave?”

15. Step-by-Step Development Plan

1. Phase 1 — Environment: Create the Python project, virtual environment, dependencies, and Git repository.
2. Phase 2 — Road Network: Build a small synthetic road graph and verify connectivity.
3. Phase 3 — Dijkstra: Implement and test shortest-path routing.
4. Phase 4 — Route Metrics: Add distance and travel-time calculations.
5. Phase 5 — Traffic: Add configurable traffic simulation to road segments.
6. Phase 6 — Energy: Implement the baseline energy model and validate calculations.
7. Phase 7 — Optimization: Normalize metrics and implement weighted route scoring.
8. Phase 8 — AI: Create traffic data, train the ML model, evaluate it, and integrate predictions.
9. Phase 9 — Simulation: Implement step-by-step e-bike movement and state tracking.
10. Phase 10 — Backend: Expose routing, optimization and simulation through FastAPI.
11. Phase 11 — Frontend: Build the map interface and controls.
12. Phase 12 — Evaluation: Run defined scenarios and record metrics.
13. Phase 13 — Documentation: Prepare report, diagrams, screenshots, test cases, results and viva explanation.

Recommended Development Rule

Do not start with the AI model. First make the graph and Dijkstra route work correctly. Then add traffic, time and energy. Finally add ML prediction and integrate it into optimization. This produces a stable baseline and makes the AI contribution easy to demonstrate.

16. Worked Simulation Scenario

Assume the simulator receives source A and destination F. It generates three candidate routes and calculates their metrics under a peak-hour traffic scenario.

Route	Distance	Time	Traffic	Energy	Interpretation
A	5 km	25 min	High	High	Shortest but congested.
B	6 km	18 min	Low	Low	Slightly longer but efficient.
C	7 km	20 min	Medium	Medium	Longer alternative.

If the normalized weighted score of Route B is lower than Routes A and C, the simulator selects Route B. The important result is not the specific numbers above, but the behavior: a route that is not the shortest can become optimal when traffic, time and energy are included.

Expected Result Presentation

- Show all candidate routes on the map.
- Highlight the AI-optimized route.
- Display route distance, travel time, traffic and energy.
- Show the simulated e-bike moving along the selected route.
- Provide a comparison chart after the simulation.
- Store the scenario so it can be repeated during demonstration.

17. Expected Project Deliverables

- Working Python backend for graph routing and simulation.
- Dijkstra baseline and optional A* implementation.
- Synthetic road and traffic datasets.
- Machine-learning traffic prediction model.
- Travel-time and energy-consumption modules.
- Multi-factor route optimizer.
- FastAPI endpoints.
- Interactive Leaflet map interface.
- Simulation controls and route comparison dashboard.
- Evaluation results for multiple scenarios.
- Project report, diagrams, screenshots, test cases and presentation.

Final System Output

The final demonstration should allow an evaluator to select a source, destination and scenario; generate candidate routes; run AI-assisted optimization; simulate the e-bike journey; and inspect the resulting distance, travel time, traffic, energy consumption and overall route score.

18. Key Viva Explanation

What is the project?

It is a software simulator that studies and optimizes e-bike movement through an urban road network under different traffic and mobility conditions.

Where is AI used?

AI/ML is used to predict traffic or travel conditions. These predictions become inputs to the route optimization process. Therefore, the project is not simply Dijkstra with an AI label.

Why not just choose the shortest route?

Because the shortest route may have heavy traffic, longer travel time or higher energy use. The simulator evaluates several factors together.

Why is it different from Google Maps?

A general navigation application is designed to guide real users. This project is an academic simulation and experimentation platform focused specifically on e-bike route efficiency, energy, traffic prediction, controlled scenarios and algorithm comparison.

What makes it a simulator?

The system can create controlled traffic and road conditions, run the same source-to-destination journey under different scenarios, simulate e-bike movement, and measure the resulting performance.

19. Conclusion

The AI Urban Micro Mobility Simulator should be developed as an integrated pipeline rather than as an ordinary map application. The graph-routing layer provides a reliable baseline, the traffic and energy modules add e-bike-specific conditions, the machine-learning model introduces the AI component, and the multi-factor optimizer converts these predictions and estimates into a route decision. Finally, the simulation engine and map interface make the behavior visible and measurable.

The recommended implementation order—graph → Dijkstra → route metrics → traffic → energy → optimization → AI prediction → simulation → visualization → evaluation—keeps the project manageable while producing a clear technical contribution.

Scope for Future Enhancement

- Real-time traffic feeds.
- Real GPS traces and trajectory data.
- Elevation-aware energy modeling.
- More advanced ML/deep-learning prediction.
- Dynamic re-routing during simulation.
- Larger road networks and realistic urban mobility datasets.
- Integration with traffic simulators for more advanced experiments.

Source/Format Note

The organization of this document follows the academic project-report style visible in the provided AITM/VTU sample: introduction, system analysis, methodology, requirements/technology, conclusion, and supporting technical detail. The sample report is for a different sign-language project, so its domain-specific content has not been reused. The project approach above is specific to the AI Urban Micro Mobility Simulator.

The provided sample report uses the AITM/VTU institutional structure and includes chapters for Introduction, Literature Survey, System Analysis, System Requirements, Conclusion and References. This PDF is intentionally a detailed project approach document rather than the final 30-page submission report.